

UF5

Frameworks para PHP I

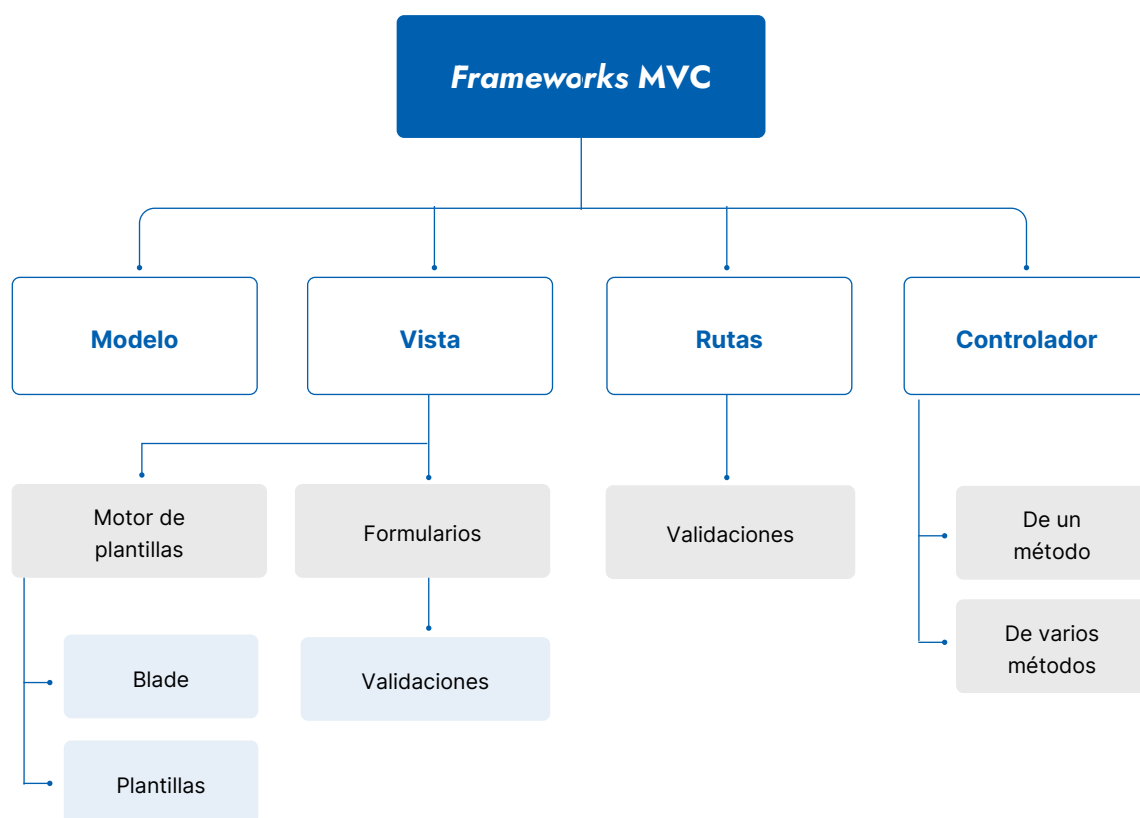
Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual	04
1. Patrón MVC. <i>Frameworks</i>	05
1.1. <i>Frameworks</i>	05
1.2. Patrón MVC (modelo-vista-controlador)	07
2. Instalación y configuración	10
2.1. Visual Studio Code	10
2.2. Apache, PHP y MariaDB/MySQL con XAMPP/LAMP	10
2.3. Laravel	10
A. Instalación de Composer	11
B. Instalación de Laravel.....	11
C. Actualización de Laravel.....	11
2.4. Node.js.....	11
2.5. Comando <i>artisan</i>	12
2.6. Estructura de un proyecto en Laravel	13
3. Aspectos básicos. Controladores y rutas.....	16
3.1. Controladores.....	17
3.2. Controladores de un solo método (<i>invoke</i>)	18
3.3. Controladores de varios métodos: controladores de recursos	19
3.4. Rutas	20
A. Rutas simples	22
B. Rutas con parámetros	22
C. Rutas con alias o named routes	22
D. Rutas con validaciones para distintos parámetros	23

4. Vistas: motor de plantillas	24
4.1. Pasar variables a las vistas	24
4.2. Motor de plantillas Blade	25
A. Estructuras de control de flujo en Blade	25
B. Enlaces a otras rutas con Blade	28
4.3. Definir plantillas con Blade	29
5. Formularios	30
5.1. Envío de formularios con Laravel.....	31
5.2. Validación de formularios.....	33
A. Form requests para validaciones más complejas.....	34
B. Mostrar mensajes de error.....	35
C. Recordar valores enviados	36

Mapa conceptual



01 Patrón MVC. Frameworks

1.1. Frameworks

Un **framework** es una plataforma o herramienta para la creación de aplicaciones web, que ayuda a organizar proyectos complejos, de manera que nos puede ahorrar mucho tiempo en la codificación del proyecto. Esto nos permitirá ahorrar tiempo y coste en la producción de aplicaciones complejas de software, tan necesarios hoy en día a nivel empresarial en un mundo tan competitivo.

Un **framework** se podría entender como un conjunto de servicios o funcionalidades, realizados de acuerdo con patrones de diseño, que, además de ofrecer funciones básicas para el desarrollo de una aplicación, mecanismos de seguridad, etcétera, obligan al programador a estructurar su código según patrones como el modelo-vista-controlador.

IMPORTANTE

Un *framework* proporciona una estructura base, a partir de la cual podemos empezar la implementación de un proyecto. Nos da facilidades para crear la conexión a la base de datos, el modelo de datos, librerías de funciones como autenticación de usuarios, etcétera. Lo importante es que tenemos muchas funcionalidades ya realizadas. No tenemos que empezar desde cero con PHP nativo, lo cual hoy en día no sería rentable, ya que el proyecto se demoraría mucho

Existe una gran variedad de *frameworks* en la mayoría de los lenguajes de programación de servidor web; los tenemos en JAVA (como **Spring**), en PYTHON (como **Django**), en C# (como **ASP.NET MVC**), en NodeJS (como **Express JS**), etc. Nosotros nos vamos a centrar en los *frameworks* **PHP**. Normalmente, la mayoría de los *frameworks* siguen el patrón **MVC (modelo-vista-controlador)**, que veremos en el punto siguiente.

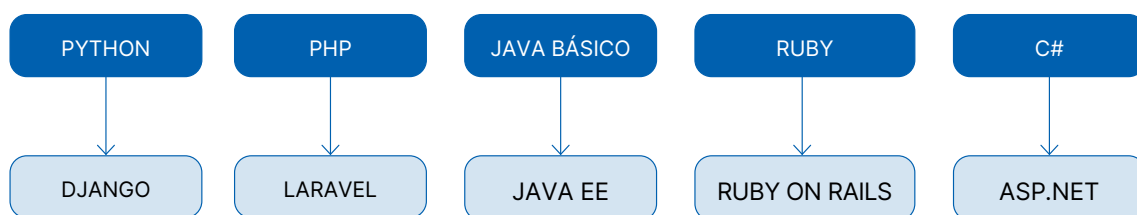


Figura 1.

PHP se usa en más del 70 % de los desarrollos web en el mundo; cabe destacar que algunos CMS como **WordPress**, **Prestashop** o **Magento** están desarrollados en este lenguaje.

LARAVEL	Creado en 2011, ha ganado mucha popularidad y parece ser que está «de moda» . Tiene una gran comunidad detrás, posee una buena documentación y dispone de librerías adicionales, como un motor de plantillas (código más legible en las vistas) u ORM (librerías de acceso a datos). Además, utiliza el <i>core</i> del <i>framework</i> Symfony (el 30 % del código de Laravel es Symfony).
SYMFONY	Creado en 2005, tiene mucho más recorrido que Laravel, con estructura más consolidada. Se instalaban demasiados módulos de serie que no necesitábamos, y actualmente tiene una estructura modular. Prestashop 1.7 está desarrollado con este <i>framework</i> .
CODEIGNITER	Creado en 2006, es de los más ligeros y sencillos de instalar. Tiene una curva de aprendizaje muy rápida, debido a su sencillez. Sufrió una etapa de abandono, pero ahora, con la versión 4, sigue pisando fuerte.
CAKEPHP	Creado en 2005, es muy similar a Codeigniter. La última versión es CakePHP 4.3 Strawberry. Hay una amplia comunidad detrás dando soporte.
ZEND	Creado en 2006, es otro <i>framework</i> muy popular, con menor visibilidad que los anteriores, pero parecido a CakePHP .
PHALCON	Creado en 2012, su principal ventaja frente a otros <i>frameworks</i> similares es su rendimiento. A diferencia de otros <i>frameworks</i> escritos en PHP, Phalcon está escrito en C y se compila agregándolo como módulo en Apache o Nginx.

Tabla 1. Ejemplos de frameworks más utilizados para PHP en el año 2022.

¿Cuál utilizar de ellos? ¿Por qué Laravel?

- » Si atendemos, por ejemplo, a las búsquedas en **Google Trends** de los *frameworks* PHP, Laravel se posiciona en primer lugar.
- » Si nos decantamos por la demanda laboral y realizamos una búsqueda de estas tecnologías en **InfoJobs**, Laravel aparece como el que más demanda laboral tiene, seguido de Symfony y Codeigniter.

FRAMEWORKS	NÚMERO DE OFERTAS ENCONTRADAS
Laravel	109
Symfony	106
Codeigniter	16

Tabla 2. Resultados obtenidos en marzo de 2022.

1.2. Patrón MVC (modelo-vista-controlador)

Como hemos comentado anteriormente, la mayoría de los *frameworks* de desarrollo web se sustentan en el patrón **MVC** (en inglés, *model-view-controller*). Es el patrón rey en el mundo del desarrollo web, y además en muchas aplicaciones de escritorio.

Este patrón se basa en dividir el diseño en tres capas fundamentales:

- » **Modelo:** estará formado por las clases o elementos donde guardaremos los datos, así como las reglas de negocio de nuestra aplicación. Es decir, todas las funcionalidades relacionadas con el acceso a datos y la gestión de estos. La mayoría de los *frameworks* disponen de mecanismos de mapeado objeto-relacional (ORM), mediante los cuales hacen una traducción directa de un modelo relacional (base de datos) a un modelo de objetos (lenguaje), simplificando mucho las operaciones.
- » **Vista:** es la presentación final que verá el usuario, páginas web, formularios, etc. Representa lo que el usuario visualizará (HTML y CSS, en nuestro caso). Recibe información del controlador para renderizarla. Esta información se compone, generalmente, de datos u objetos del modelo.
- » **Controlador:** se encarga de coordinar y controlar el funcionamiento general de la aplicación. Desde la petición del usuario hasta las llamadas al modelo, la recuperación de datos y la presentación de la vista al usuario. Se podría decir que hace de intermediario entre la vista y el modelo. Recibe eventos u órdenes del usuario (en un *framework* web, serían las peticiones HTTP), consultando o modificando el modelo, y envía la información necesaria a la vista para que esta se genere.

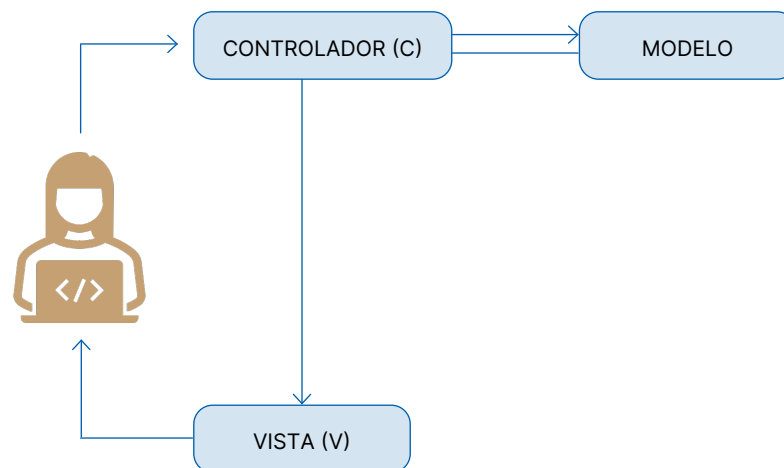
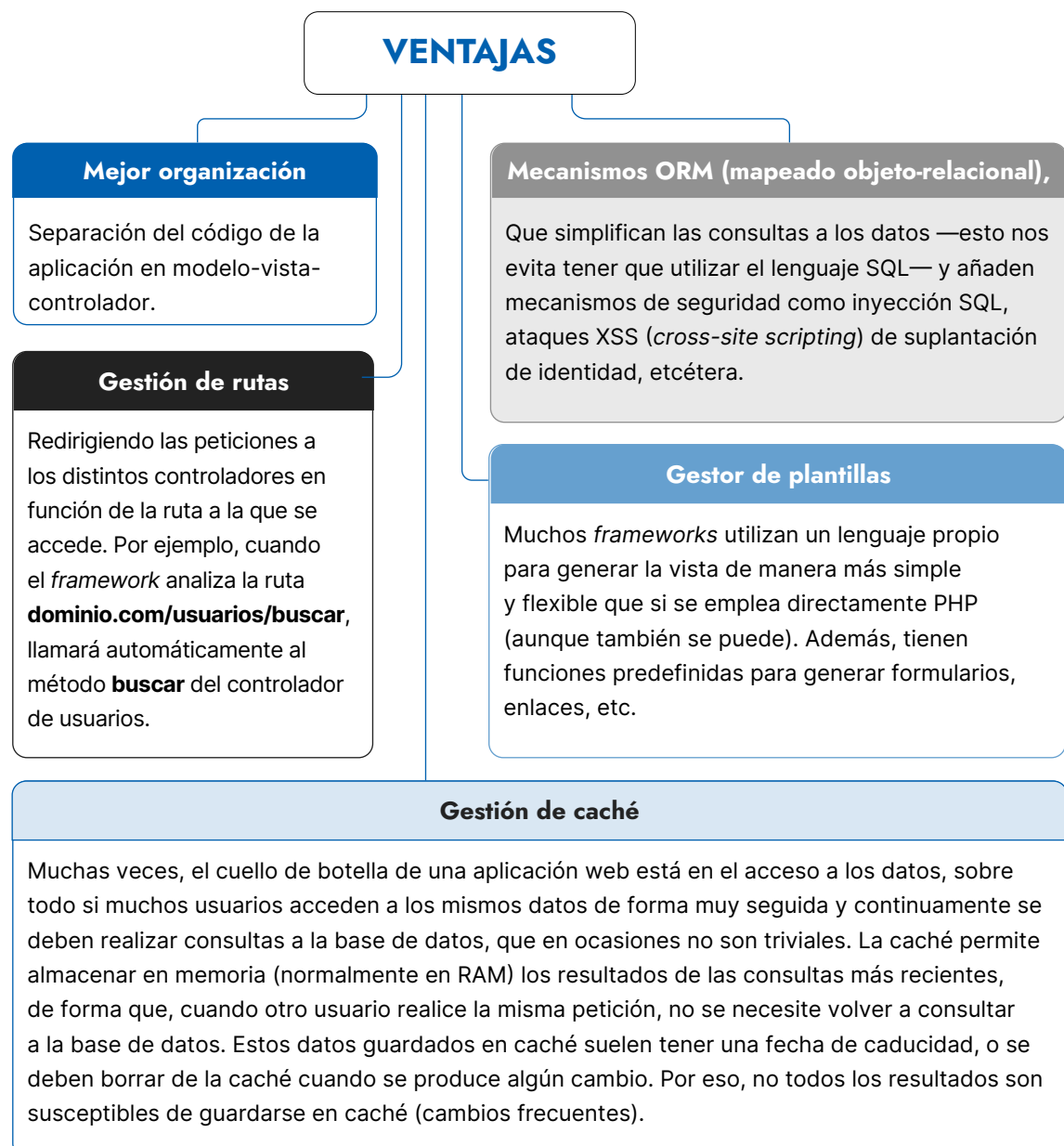
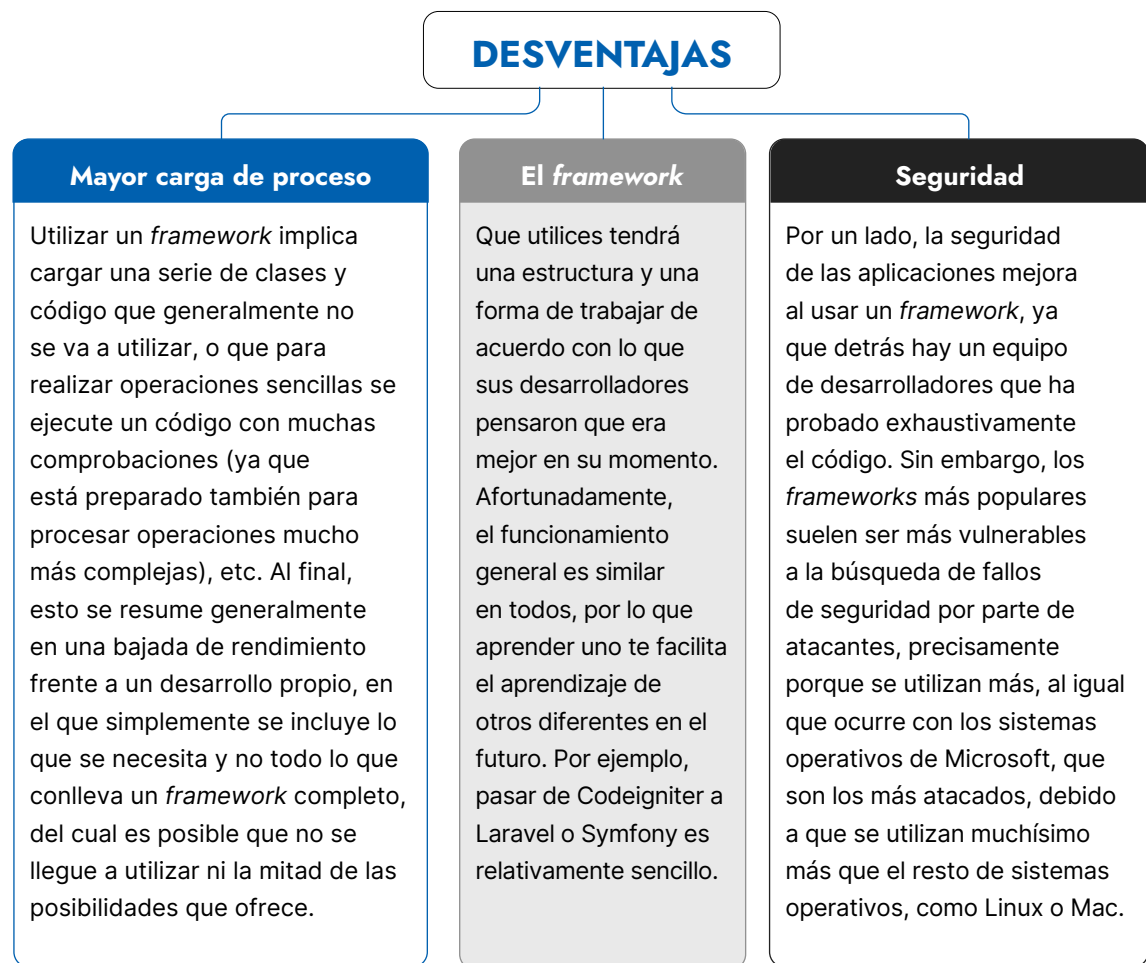


Figura 2. Interacción entre un usuario y la arquitectura MVC.





Esquema 1. Pros y contras del patrón MVC.

En resumen, la utilización de un *framework* suele ofrecer muchas más ventajas que inconvenientes, y esto se acentúa conforme la aplicación que haya que desarrollar sea mayor y más compleja. La cuestión es saber qué *framework* se ajusta mejor a nuestras necesidades.

¿Por qué se utiliza un *framework*?

Desarrollo rápido y menos código que escribir.

Bibliotecas comunes a los proyectos ya realizados.

Código muy bien estructurado y organizado.

Incluyen muchos aspectos de seguridad que no tenemos que implementar.

Gran facilidad en el manteniendo y trabajo en equipo.

02 Instalación y configuración

En este apartado vamos a instalar todo el software necesario para poder crear una aplicación web desarrollada en **Laravel**. Analizaremos los pasos necesarios para instalar el entorno de ejecución en un sistema Linux basado en Debian o en Ubuntu o un sistema Windows. Para empezar, a lo largo de estas unidades realizaremos todos los ejercicios en Local.

Herramientas necesarias para instalar y desarrollar una aplicación web en Laravel.

- » Editor de código en **Visual Studio Code**.
- » Sistema **XAMPP** o **LAMP** (se supone que ya se tiene instalado de las unidades anteriores), y además el **phpMyAdmin**.
- » El *framework* Laravel.
- » **Node.js**, ya que necesitaremos dependencias para la parte cliente.

2.1. Visual Studio Code

Es un IDE muy configurable para desarrollos web; tiene un gran repositorio de *plugins*.

Lo vamos a descargar desde la web oficial para la plataforma deseada, <https://code.visualstudio.com/>

Una vez instalado, vamos a la sección de *Extensiones*, del menú izquierdo del IDE; buscamos la extensión **Laravel Extension Pack** e instalamos esa extensión.

Tenemos un terminal integrado en la parte inferior del editor, **ver-> Terminal**. Este terminal lo vamos a utilizar muy a menudo.

2.2. Apache, PHP y MariaDB/MySQL con XAMPP/LAMP

Partimos de que ya tenemos todo el entorno instalado, PHP con versión mínima 8.x, MariaDB o MySQL, y el administrador gráfico de bases de datos MariaDB o MySQL **phpMyAdmin**.

2.3. Laravel

Para trabajar con Laravel, tendremos que instalar la utilidad **Composer** (gestor de dependencias y utilidad para crear proyectos Laravel).

A. Instalación de Composer

La instalación de Laravel se realiza a través del gestor de dependencias **Composer**.

Vamos a instalar **Composer** de manera global; de esta forma, podremos gestionar varios proyectos en el mismo equipo.

Lo descargamos de la página oficial (<https://getcomposer.org/download/>); seguimos las instrucciones, dependiendo de la plataforma que se vaya a instalar.

Al final, descargaremos el archivo `composer.phar`, lo renombraremos a `composer` y lo copiaremos en un directorio del PATH del sistema, dándole permisos de ejecución para convertirlo a un ejecutable; de esta manera, podemos ejecutarlo desde cualquier directorio.

Puedes encontrar más información sobre **Composer** en el archivo **¿Qué es Composer?**, en la sección **Descarga de documentos** de la unidad.

B. Instalación de Laravel

```
composer global require laravel/installer
```

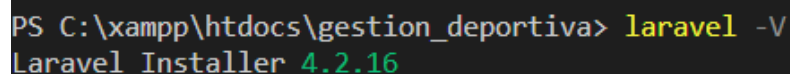
C. Actualización de Laravel

Con el comando anterior, instalamos la última versión de Laravel; no obstante, con el paso del tiempo, conviene ir actualizando la versión de Laravel.

```
composer global update laravel/installer
```

Comprobamos la versión del instalador de Laravel:

```
laravel -V
```



```
PS C:\xampp\htdocs\gestion_deportiva> laravel -V
Laravel Installer 4.2.16
```

Figura 3.

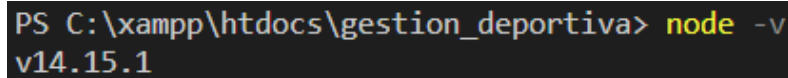
2.4. Node.js

Al instalar Node.js se instala la herramienta **NPM** (Node Package Manager), herramienta que permite instalar librerías de JavaScript, como Bootstrap o jQuery.

Lo descargamos desde la página oficial, <https://nodejs.org/es/download/>.

Una vez instalado, podemos comprobar la versión de **Node.js**.

`node -v`



```
PS C:\xampp\htdocs\gestion_deportiva> node -v
v14.15.1
```

Figura 4.

IMPORTANTE

Para comprobar que lo hemos instalado todo correctamente, podemos crear un proyecto de prueba en Laravel. Ejecutamos la siguiente instrucción dentro de una carpeta de pruebas:

`laravel new Gestion`

Con esto, se debería crear un proyecto llamado **Gestion**.

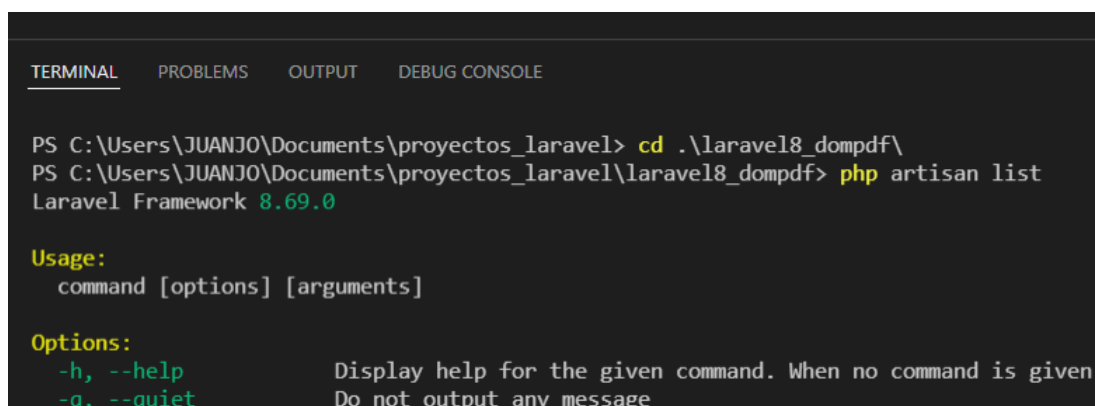
2.5. Comando *artisan*

Cuando creamos un proyecto en Laravel, automáticamente se instala una herramienta llamada **artisan** en la raíz del proyecto. Es una interfaz de línea de comandos para gestionar los proyectos Laravel, crear modelos, migraciones, controladores, etc.

Podemos comprobar que está instalada correctamente.

Vamos a la terminal del Visual Studio Code, y ejecutamos desde la raíz del proyecto:

`php artisan list`



```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

PS C:\Users\JUANJO\Documents\proyectos_laravel> cd .\laravel8_dompdf\
PS C:\Users\JUANJO\Documents\proyectos_laravel\laravel8_dompdf> php artisan list
Laravel Framework 8.69.0

Usage:
  command [options] [arguments]

Options:
  -h, --help                Display help for the given command. When no command is given
  -q, --quiet                Do not output any message
```

Figura 5.

Con el comando `php artisan list` obtenemos un listado de todas las opciones que nos ofrece **artisan**.

Para comprobar la versión de **artisan**:

`php artisan --version`

2.6. Estructura de un proyecto en Laravel

Al crear un nuevo proyecto de Laravel se nos generará una estructura de carpetas y ficheros para organizar nuestro código. Es importante que conozcamos para qué vale cada elemento y dónde tenemos que colocar nuestro código. En este apartado lo iremos viendo poco a poco, por lo que más adelante se volverán a explicar algunos de estos elementos más en detalle. Pero de momento vamos a explicar brevemente las carpetas que más utilizaremos y las que mejor tendremos que conocer:

- » **app** – contiene el código principal de la aplicación. Esta carpeta, a su vez, está dividida en muchas subcarpetas que analizaremos en la siguiente sección.
- » **config** – aquí se encuentran todos los archivos de configuración de la aplicación: base de datos, caché, correos, sesiones o cualquier otra configuración general de la aplicación.
- » **database** – en esta carpeta se incluye todo lo relacionado con la **definición de la capa de acceso a datos** de nuestro proyecto. Dentro de ella podemos encontrar a su vez tres carpetas: **factores**, **migrations** y **seeds**. En el apartado sobre bases de datos analizaremos mejor su contenido.
- » **public** – es la única carpeta pública, la única que debería ser **visible** en nuestro servidor web. Todas las peticiones y solicitudes a la aplicación pasan por esta carpeta, ya que en ella se encuentra **index.php**; este archivo es el que inicia todo el proceso de ejecución del *framework*. En este directorio también se alojan los archivos CSS, JavaScript, imágenes y otros archivos que se quieran hacer públicos.
- » **resources** – esta carpeta contiene a su vez tres carpetas: **assets**, **views** y **lang**:
 - **resources/views** – este directorio contiene las vistas de nuestra aplicación. En general serán plantillas de HTML que usan los controladores para mostrar la información. Hay que tener en cuenta que en esta carpeta no se almacenan los JavaScript, CSS o imágenes; ese tipo de archivos se tienen que guardar en la carpeta **public**.
 - **resources/lang** – en esta carpeta se guardan archivos PHP que contienen *arrays* con los textos de nuestro sitio web en diferentes lenguajes; solo será necesario utilizarla en caso de que se desee que la aplicación se pueda traducir.
 - **resources/assets** – se utiliza para almacenar los fuentes de los *assets* tipo *less* o *sass* que se tendrían que compilar para generar las hojas de estilo públicas. No es necesario usar esta carpeta, ya que podemos escribir directamente las hojas de estilo dentro de la carpeta **public**.

- » **bootstrap** – en esta carpeta se incluye el código que se carga para procesar cada una de las llamadas a nuestro proyecto. Normalmente, no tendremos que modificar nada de esta carpeta.
- » **routes** – en esta carpeta almacenamos las rutas de la aplicación (**web.php**).
- » **storage** – en esta carpeta, Laravel almacena toda la información interna necesaria para la ejecución de la web, como son los archivos de sesión, la caché, la compilación de las vistas, metainformación y los *logs* del sistema. Normalmente, tampoco tendremos que tocar nada dentro de esta carpeta; únicamente se suele acceder a ella para consultar los *logs*.
- » **tests** – esta carpeta se utiliza para los ficheros con las pruebas automatizadas. Laravel incluye un sistema que facilita todo el proceso de pruebas con PHPUnit.
- » **vendor** – en esta carpeta se alojan todas las librerías y dependencias que conforman el *framework* de Laravel. Esta carpeta tampoco la tendremos que modificar, ya que todo el código que contiene son librerías que se instalan y actualizan mediante la herramienta Composer.

Además, en la carpeta raíz también podemos encontrar tres ficheros muy importantes y que también utilizaremos:

- » **.env** – este fichero ya lo hemos mencionado en la sección de instalación; se utiliza para almacenar los valores de configuración que son propios de la máquina o instalación actual, lo que nos permite cambiar fácilmente la configuración según la máquina en la que se instale y tener opciones distintas para producción, para distintos desarrolladores. Importante: este fichero debería estar en el **.gitignore**.
- » **composer.json** – este fichero es el utilizado por Composer para realizar la instalación de Laravel. En una instalación inicial únicamente se especificará la instalación de un paquete (el propio *framework* de Laravel), pero podemos especificar la instalación de otras librerías o paquetes externos que añadan funcionalidad a Laravel. En este archivo van las dependencias PHP.
- » **package.json** – en este archivo se encuentran algunas dependencias para la parte cliente (Bootstrap o JQuery), y se encuentran preinstaladas en la carpeta **node_modules**.

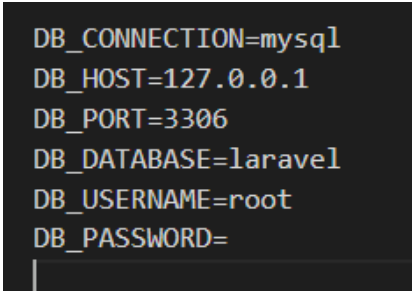
Estos tres archivos no deben subirse a ningún repositorio (por ejemplo, GitHub), ya que, si importamos un proyecto en Laravel, podemos regenerar las dependencias de PHP con **composer install** y las dependencias de JavaScript con **npm install**; es decir, los archivos **composer.json** y **package.json** actúan como índice de dependencias de PHP y JavaScript, respectivamente.

El primer fichero que deberíamos configurar es el archivo `.env`, situado en la raíz del proyecto.

El archivo de configuración `.env`, presente en la raíz del proyecto, incluye una sección en la que deberemos configurar los parámetros de conexión a la base de datos.

```
APP_NAME = Nombre del proyecto
...
DB_CONNECTION = mysql
DB_HOST = 127.0.0.1
DB_PORT = 3306
DB_DATABASE=nombre de la base de datos
DB_USERNAME=usuario de la base de datos
DB_PASSWORD=password de la base de datos
```

Ejemplo para un entorno XAMPP en Windows en local con base de datos `laravel` y usuario `root` y sin contraseña:



```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

Figura 6.

Para probar nuestros proyectos, en principio, además de configurar un servidor web (como Apache), tendríamos que dar previamente los permisos necesarios a ciertas carpetas; por ejemplo, para un entorno Linux de pruebas en local. Evidentemente, para un entorno en producción utilizaremos permisos más restrictivos, para que permitan al usuario `apache` leer o escribir en ellos.

```
$ sudo chmod -R 777 bootstrap/cache
$ sudo chmod -R 777 storage
```

Dentro de Laravel, ya tenemos un servidor interno que haría la función de servidor web: el propio Laravel, mediante el comando `artisan`, nos permite poner en marcha la aplicación web prescindiendo del **servidor web Apache**.

Desde la raíz del proyecto, ejecutaremos el siguiente comando:

```
php artisan serve
```

```
view:cache      Compile all of the application's Blade templates
view:clear      Clear all compiled view files
PS C:\Users\JUANJO\Documents\proyectos_laravel\laravel8_dompdf> php artisan serve
Starting Laravel development server: http://127.0.0.1:8000
```

Figura 7.

Esto pondrá en marcha un servidor por el puerto 8000; podremos acceder a nuestro proyecto a través de la URL

<http://localhost:8000>

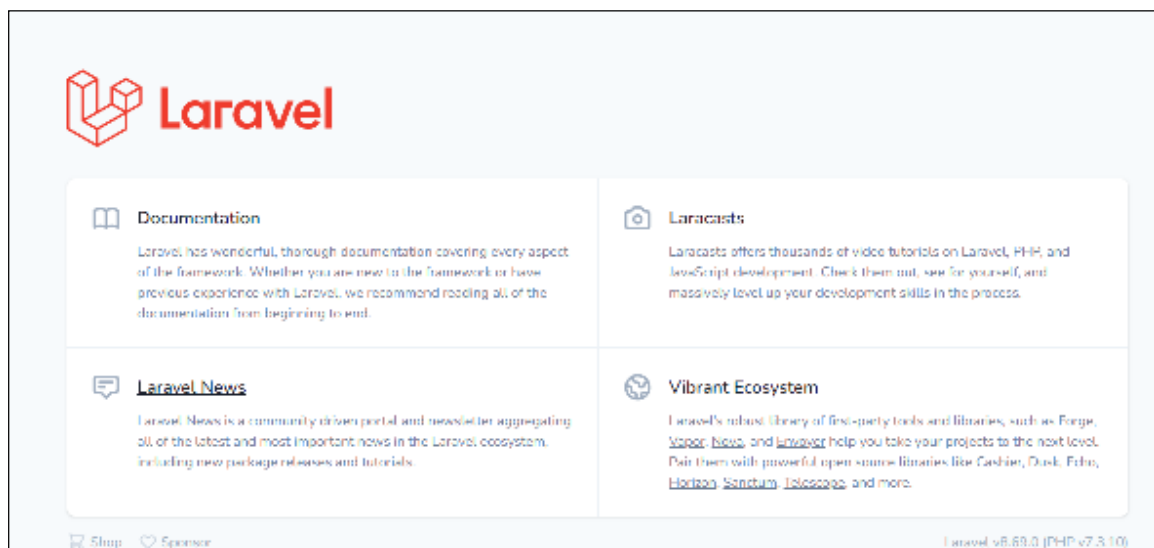


Figura 8.

Si hemos llegado hasta aquí, eureka, hemos triunfado.

03 Aspectos básicos. Controladores y rutas

Una vez vista la estructura de carpetas y archivos al crear un proyecto en Laravel, vamos a explicar dos elementos importantes en la gestión de un proyecto Laravel: controladores y rutas.

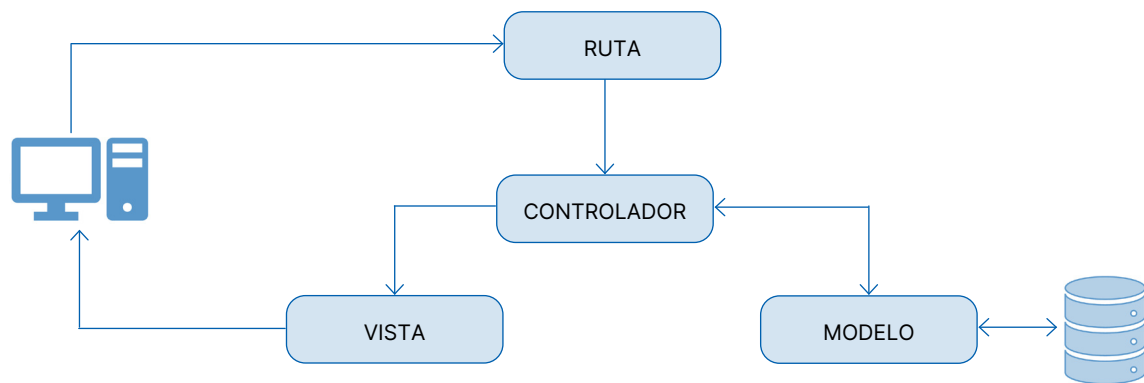


Figura 11. Estructura de Laravel.

3.1. Controladores

Un controlador es el componente que nos permite agrupar, de una mejor manera, el código necesario para dar respuesta a una petición HTTP; es el intermediario entre la vista y el modelo y se encarga de definir la lógica de negocio y los controles necesarios para resolver las solicitudes que llegan.

IMPORTANTE

Para crear un nuevo controlador, ejecutamos el siguiente comando con Artisan:

```
$ php artisan make:controller ClienteController
```

Crearé dentro de la carpeta `app\Http\Controllers` una clase `ClienteController` con el siguiente contenido:

```
<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;

class ClienteController extends Controller
{
}
```

Como podemos observar, la nueva clase creada, heredera de `Controller`, es una clase del núcleo de Laravel que agrupa los métodos y atributos comunes a todos los controladores. Todas estas clases parten de un espacio de nombres común llamado `Illuminate`, por lo que será frecuente encontrar instrucciones `use Illuminate\Xxxx`.

Creemos un método llamado `alta`, que lo único que haga es devolver un texto.

```
class ClienteController extends Controller
{

    public function alta()
    {
        return "Llamando al método alta() desde el controlador Cliente";
    }
}
```

3.2. Controladores de un solo método (*invoke*)

El comando **artisan** anterior admite algunos parámetros adicionales más. Uno muy útil es el parámetro **-i**, que crea el controlador con un método llamado **invoke**, que se autoejecuta cuando es llamado desde algún proceso de enrutamiento. Por ejemplo, si creamos el controlador así:

```
php artisan make:controller PruebaController -i
```

se creará la clase **PruebaController** en la carpeta **app/Http/Controllers**, con un contenido como este:

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class PruebaController extends Controller
{
    ...
    public function _invoke(Request $request)
    {
        ...
    }
}
```

Dentro del método **invoke**, podemos definir la lógica de generar u obtener los datos que necesita una vista, y renderizarla. Por ejemplo:

```
public function invoke(Request $request)
{
    $datos = array(...);
    return view('miVista', compact('datos'));
}
```

Las rutas en Laravel son uno de los componentes más importantes y potentes del *framework*; se encargan de manejar el flujo de solicitudes HTTP, desde y hacia el cliente; las peticiones realizadas por el navegador suelen ser **get**, **post**, **put**, **delete** y **patch**, de HTTP hacia una URL concreta.

Así, en el archivo de rutas, basta con definir la ruta que queramos, y, como segundo parámetro del método **get**, indicar el nombre del controlador que se va a disparar para procesar esa ruta. Adicionalmente, también le podemos asignar un nombre o alias a la ruta.

```
Route::get('prueba', PruebaController::class)->name('prueba');
```

3.3. Controladores de varios métodos: controladores de recursos

Si creamos un controlador con la opción **-r** en lugar de la opción **-i** utilizada en el ejemplo anterior, creará un controlador de recursos (**resources**), y predefinirá en él una serie de métodos de utilidad vacíos para cada una de las operaciones principales que se pueden realizar sobre una entidad de nuestra aplicación:

- » **index** – muestra un listado de los elementos de esa entidad o recurso.
- » **create** – muestra el formulario para dar de alta nuevos elementos.
- » **store** – almacena en la base de datos el recurso creado con el formulario anterior.
- » **show** – muestra los datos de un recurso específico (a partir de su clave o **id**).
- » **edit** – muestra el formulario para editar un recurso existente.
- » **update** – actualiza en la base de datos el recurso editado con el formulario anterior.
- » **destroy** – elimina un recurso por su identificador.

Obviamente, el código de todos estos métodos aparecerá vacío al principio, y los deberemos rellenar con las operaciones correspondientes más adelante.

Si queremos utilizar un controlador de este tipo, y llamar a alguno de sus métodos desde alguna ruta, ya no basta con poner el nombre del controlador, como hacíamos antes con los de tipo **invoke**, puesto que ahora hay más de un método que elegir. Lo que haremos será crear un array en el que el primer elemento es la clase del controlador, y el segundo elemento es el nombre del método que se va a invocar. Por ejemplo:

```
Route::get('prueba', [PruebaController::class, 'index'])->name('listado_prueba');
```

Vamos a probar esta opción para crear un controlador de **Proveedores**. Crearemos un controlador para gestionar los proveedores con sus altas, bajas y modificaciones con este comando:

```
php artisan make:controller -r ProveedorController
```

De momento, varios de los métodos generados en el controlador no los vamos a utilizar. Podemos modificar los dos que sí vamos a usar de momento, `index` y `show`. Así nos quedarían, respectivamente:

```
public function index()
{
    $clientes = array(
        array(
            "nombre" => "Juan",
            "apellidos" => "Gutiérrez Sanz"
        ),
        array(
            "nombre" => "Vicente",
            "apellidos" => "Sánchez Medina"
        ),
        array(
            "nombre" => "Pablo",
            "apellidos" => "Pérez Álvarez"
        ),
    );

    return view('listado', compact('clientes'));
}

public function show($id)
{
    return "Mostrando ficha de Proveedor $id";
}
```

3.4. Rutas

Con las rutas, construimos las URL amigables (son fáciles de recordar y más seguras) de nuestra aplicación, muy importantes —entre otras cosas— para el posicionamiento web. Son un mecanismo que nos permite establecer el controlador al que debemos enviar la petición de una determinada URL. Por ejemplo, sea la siguiente URL:

```
https://127.0.0.1:8000/cliente/alta
```

Esta URL está llamando al controlador `cliente` y método `alta`. El método `alta` no es más que un método de la clase `Cliente`, y la clase `Cliente` es un controlador o una clase hija que hereda de la clase base de Laravel `Controller`.

Se configuran dentro de la carpeta `routes`, en dos archivos; básicamente: en `web.php`, que nos permiten cargar vistas (contenido típicamente **HTML**) en función de la URL, y en el archivo `api.php`, el cual nos permite definir servicios **API REST**, que veremos en apartados y unidades posteriores.

En el archivo `routes/web.php` definiremos las rutas web. Inicialmente ya existe una ruta predefinida a la raíz del proyecto.

```
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});
```

Lo que hace dicha ruta es llamar al método `view`, que carga una vista o archivo final HTML situado en `resources/views/welcome.blade.php`. Veremos en próximos apartados y unidades la extensión `blade` (motor de plantillas); ahora simplemente debes quedarte con que el producto final es un documento HTML.

Para definir una ruta, realizamos una llamada al método estático `get` de la clase `Route`, `Route::get`.

Vamos a añadir, como ejemplo, otra ruta mediante la cual nos muestre nuestro propio mensaje de bienvenida.

```
Route::get('/hola', function() {
    return "Hola, es mi primera ruta";
});
```

Probamos la ruta

http://127.0.0.1:8000/hola

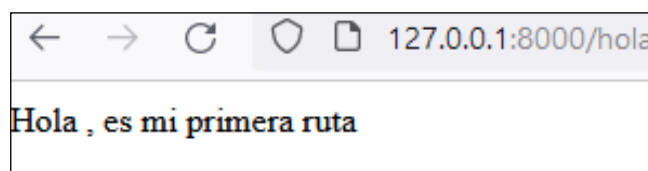


Figura 12.

Hay que recordar que para poder probar nuestra aplicación Laravel hemos de ejecutar `php artisan serve` desde la terminal de nuestro IDE Visual Studio Code.

Vamos a profundizar en el tipo de rutas:

A. Rutas simples

Es la que hemos definido anteriormente: un nombre de ruta fijo asociado a una función.

Nombre de ruta `hola` asociada a la función `function() {}`.

B. Rutas con parámetros

Podemos pasar parámetros a las rutas. Incluimos el nombre del parámetro entre llaves, y lo pasamos a la función como argumento.

En caso de parámetro obligado.

```
Route::get('hola/{nombre}', function($nombre) {  
    return "Hola, $nombre";  
});
```

En caso de parámetros opcionales, lo indicamos con un interrogante, pero debemos de poner un parámetro por defecto en la función, para evitar errores en la salida; en caso de no ponerlo, asignaría ese parámetro por defecto.

```
Route::get('hola/{nombre?}', function ($nombre = "Anónimo") {  
    return "Hola, $nombre";  
})->where('nombre', "[A-Za-z]+");
```

Para algunos parámetros sí necesitaremos que sigan un determinado patrón, que solo contenga letras, dígitos, etcétera; podemos emplear el método **where** al definir la ruta. Este método contiene el nombre del parámetro que hay que validar y la expresión regular que tiene que cumplir.

```
Route::get('hola/{nombre?}', function ($nombre = "Anónimo") {  
    return "Hola, $nombre";  
})->where('nombre', "[A-Za-z]+");
```

C. Rutas con alias o named routes

Al definir la ruta, asociamos al método **name** el nombre que queramos. Como veremos posteriormente, esto es interesante cuando esta ruta forma parte de algún enlace en alguna página de nuestra aplicación.

```
Route::get('clientes', function () {  
    return "Listado de clientes";  
})->name('ruta_clientes');
```

D. Rutas con validaciones para distintos parámetros

Es la que hemos definido anteriormente:

```
Route::get('hola/{nombre?}/{id?}',
function($nombre="Invitado", $id=0)
{
return "Hola, $nombre. Tu código es el $id";
})->where('nombre', "[A-Za-z]+")
->where('id', "[0-9]+")
->name('saludo');
```

Si accedemos a las siguientes URL, obtendremos cada una de las respuestas indicadas:

URL	SALIDA
/hola	Hola, Invitado. Tu código es el 0
/hola/Juanjo	Hola, Juanjo. Tu código es el 0
/hola/Juanjo/25	Hola, Juanjo. Tu código es el 25
/hola/45	Error 404 (URL incorrecta)

Tabla 1. Diferentes salidas en función de la URL especificada.

Notemos que el último caso es incorrecto. No podemos especificar un **id** sin haber especificado un nombre delante, porque incumple el patrón de la URL. Se puede dejar un parámetro omitido, siempre y cuando los posteriores también lo estén.

Las **rutas en Laravel** son el componente más **importante** y poderoso del *framework*; se encargan de manejar el flujo de solicitudes HTTP, desde y hacia el cliente; las peticiones realizadas por el navegador son, en lo general, **get**, **post**, **put**, **delete**, **patch** de HTTP hacia una URL concreta.

04 Vistas: motor de plantillas

Hemos visto hasta ahora que las rutas devuelven texto a través de la instrucción `return` o contenido en HTML a través de la función `view` de Laravel. **View** llama a una vista o página PHP (situada en la carpeta `/resources/views`), la cual se renderiza y se transforma en código HTML.

Laravel nos permite separar de la lógica de la aplicación las vistas de presentación al usuario final; las vistas pueden contener código HTML, CSS y PHP.

De forma predeterminada, en la carpeta `resources/views` tenemos una vista de ejemplo llamada `welcome.blade.php`. Observamos que tiene una extensión `blade.php` (esto hace referencia al motor de plantillas Blade).

4.1. Pasar variables a las vistas

Es muy normal pasar información como variables desde las rutas o los controladores a las vistas. Por ejemplo, pasar listados de datos.

En este ejemplo estamos pasando la variable `nombre` y `apellidos` a la vista `inicio`, con el método `with`:

```
Route::get('/', function() {  
    $nombre = "Pepe";  
    $apellidos = "Pérez";  
    return view('inicio')->with(['nombre' => $nombre , 'apellidos' =>  
$apellidos ] );  
});
```

Otra alternativa es pasarlo como segundo parámetro a la función `view` como un *array* asociativo.

```
return view('inicio', ['nombre' => $nombre , 'apellidos' => $apellidos ] );
```

Otra mucho más usada es con la función `compact` como segundo parámetro de `view`. En este caso, solo le pasamos el nombre de la variable definida en nuestro código cuyo valor queremos pasar a la vista.

```
return view('inicio', compact('nombre', 'apellidos'));
```

En este ejemplo estamos pasando las variables `nombre` y `apellidos` a la vista `inicio`, de forma que podemos acceder a ellas como si de una variable local se tratara.

```
<p>bienvenido a <?php echo $nombre.">".$apellidos ?></p>
```


Lo habitual es acceder a la variable mediante la sintaxis específica de Blade, el motor de plantillas de Laravel que vamos a ver a continuación:

```
<p>bienvenido a {{ $nombre }},{{ $apellidos }} </p>
```

4.2. Motor de plantillas Blade

Un motor de plantillas facilita la manera de separar la aplicación lógica y el contenido en la presentación.

Permite tener un código más legible dentro de las vistas. Por ejemplo, para mostrar una variable dentro de una vista, en vez de poner `<?php echo $variable ?>` ponemos `{{ $variable }}`.

Laravel cuenta con un motor de plantillas llamado **Blade** para evitar introducir código **PHP nativo**, introduciendo un lenguaje alternativo en nuestro código **HTML** de una manera sencilla y segura. Con esto podemos crear y separar en trozos de código que luego se pueden aprovechar en distintas vistas de la aplicación. Todo esto nos permite simplificar el desarrollo y mantenimiento de las vistas conservando la separación entre las distintas capas del patrón modelo-vista-controlador.

Empezamos a trabajar con las vistas; lo primero que haremos será crear la carpeta clientes **en la ruta `resources/views`**.

A. Estructuras de control de flujo en Blade

Todas las directivas en **Blade** van precedidas del símbolo `@`. Aquí mostramos las principales directivas:

a) Condicional si:

```
@if (count($records) === 1)
    ;Tengo un registro!
@elseif (count($records) > 1)
    ;Tengo varios registros!
@else
    ;Yo no tengo ningún registro!
@endif
```

Estructuras de control `@foreach` / `@forelse`.

Para iterar sobre un conjunto de datos (*array*), podemos emplear la directiva `@foreach`, con una sintaxis similar al **foreach** de PHP, pero sin necesidad de llaves. Basta con finalizar el bucle con la directiva `@endforeach`, de este modo:

```
@for ($i = 0; $i < 10; $i++)
```

```

        El valor actual es {{ $i }}
    @endfor
    @foreach ($users as $user)
        <p>Este es el usuario {{ $user->id }}</p>
    @endforeach
    @foreach ($users as $user)
        <li>{{ $user->name }}</li>
    @empty
        <p>No users</p>
    @endforeach
    @while (true)
        <p>Estoy en un bucle para siempre.</p>
    @endwhile

```

En el caso de querer realizar alguna comprobación (por ejemplo, si el *array* anterior está vacío, para mostrar un mensaje pertinente), usamos la directiva `@if`, cerrada por su correspondiente pareja `@endif`. Opcionalmente, se puede intercalar una directiva `@else` para el camino alternativo, o también `@elseif` para indicar otra condición. El ejemplo anterior podría quedar así:

```

@if ($elementos)
    @foreach ($elementos as $elemento)
        <li>{{ $elemento }}</li>
    @endforeach
@else
    <li>No hay elementos que mostrar</li>
@endif

```

También podemos comprobar si una variable está definida. En este caso, reemplazamos la directiva `@if` por `@isset`, con su correspondiente cierre `@endisset`.

```

@isset($elementos)
    @foreach($elementos as $elemento)
        <li>{{ $elemento }}</li>
    @endforeach
@else
    <li>No hay elementos que mostrar</li>
@endisset

```

Sin embargo, con cualquiera de estas opciones tenemos un problema: en el primer caso, si la variable `$elementos` no está definida, mostrará un error de PHP. En el segundo, si la variable sí está definida pero no contiene elementos, no se mostrará nada por pantalla.

Una tercera estructura alternativa que agrupa estos dos casos (controlar a la vez que la variable esté definida y tenga elementos) es emplear la directiva `@forelse` en lugar de `@foreach`. Esta directiva permite una cláusula adicional `@empty` para indicar qué hacer si la colección no tiene elementos o está sin definir. El ejemplo anterior quedaría más abreviado:

```
@forelse($elementos as $elemento)
<li>{{ $elemento }}</li>
@empty
<li>No hay elementos que mostrar</li>
@endforelse
```

En este tipo de iteradores (`@foreach` o `@forelse`), tenemos disponible un objeto llamado `$loop`, con una serie de propiedades sobre el bucle que estamos iterando, `index` (posición dentro del *array* por la que vamos), o `count` (total de elementos), o `first` y `last` (booleanos que determinan si es el primer o último elemento, respectivamente), entre otros. Podemos ver todas las propiedades disponibles en este objeto llamando a `var_dump`:

```
<ul>
@forelse($elementos as $elemento)
<li>{{ $elemento }} {{ var_dump($loop) }} </li>
@empty
<li>No hay elementos que mostrar</li>
@endforelse
</ul>
```

Si, por ejemplo, queremos determinar si es el último elemento de la lista, y mostrar un mensaje o estilo especial, podemos hacer algo como esto dentro del `@forelse`:

```
{{ $loop->last ? "Último elemento" : "" }}
```

Existen otros tipos de estructuras iterativas y selectivas en Blade, como `@while`, `@for` o `@switch`, y algunos otros. Se puede consultar en la documentación oficial de Blade: <https://laravel.com/docs/9.x/blade>

Para introducir cualquier directiva en **Blade**, hay que anteponer siempre el carácter `@`; por ejemplo, `@if`, `@endif`, `@foreach`, `@endforeach`, `@forelse`, `@endforelse`, etcétera. Para mostrar cualquier variable podemos emplear una sintaxis de dobles llaves, facilitada por **Blade**.

Siguiendo nuestro ejemplo del proyecto **gestion**, definiremos una ruta para sacar un listado de clientes; de momento, vamos a crear manualmente ese listado en el propio método de enrutamiento, y se lo pasaremos a una vista llamada `listado.blade.php`. Por un lado, la nueva ruta para el listado puede quedar así:

```
Route::get('listado', function () {
    $clientes = array(
        array(
            "nombre" => "Juan",
            "apellidos" => "Gutiérrez Sanz"
        ),
    );
    return view('listado', compact('clientes'));
});
```

Por su parte, la vista `listado.blade.php` puede quedar así:

```
<h1>Listado de clientes</h1>
<ul>
    @foreach ($clientes as $cliente)
        <li>{{ $cliente['nombre'] }} ({{ $cliente['apellidos'] }})</li>
    @empty
        <li>No se encontraron clientes</li>
    @endforeach
</ul>
```

B. Enlaces a otras rutas con Blade

Hemos comentado brevemente en puntos anteriores que, gracias a Blade y a los nombres en las rutas, podemos enlazar una vista con otra de dos formas: de forma tradicional...

```
echo '<a href="/contacto">Contacto</a>';
```

... o bien empleando la función `route`, seguida del nombre que le hemos dado a la ruta:

```
<a href="{{ route('ruta_contacto') }}">Contacto</a>
```

Además, mediante Blade existe una tercera forma de enlazar, empleando la función `url`, que genera una URL completa hasta la ruta que indiquemos:

```
<a href="{{ url('/contacto') }}">Contacto</a>
```

4.3. Definir plantillas con Blade

A la hora de dar homogeneidad a una web, es habitual que la cabecera, el menú de navegación o el pie de página formen parte de una plantilla que se repite en todas las páginas del sitio, de modo que evitamos tener que actualizar todas las páginas ante cualquier posible cambio en estos elementos.

Para crear una plantilla en Blade, creamos un archivo (por ejemplo, `plantilla.blade.php`) en la carpeta de vistas, con el contenido general de la plantilla. En aquellas zonas del documento donde vamos a permitir contenido variable dependiendo de la vista en sí, añadimos una sección llamada `@yield`, con un nombre asociado. Nuestra plantilla podría ser esta (notemos que se permiten varios `@yield` con diferentes nombres):

```
<head>
    <title>@yield('titulo')</title>
</head>
<body>
    <nav>
        <!-- ... Menú de navegación -->
    </nav>
    @yield('contenido')
</body>
</html>
```

Luego, en cada vista en la que usemos esta plantilla, añadimos la directiva `@extends`(el nombre de la plantilla). Con la directiva `@section`(nombre de la sección) añadimos el contenido de esa sección para cada uno de los `@yield` de la plantilla. Cada sección la finalizaremos con la directiva `@endsection`. Nuestra página inicial (`inicio.blade.php`) podría quedar así.

```
@extends('plantilla')
@section('titulo', 'Inicio')
@section('contenido')
    <h1>Página de inicio</h1>
    Bienvenido {{ $nombre }}
@endsection
```

Como se puede observar, a la directiva `@section` se le puede pasar un segundo parámetro con el contenido de esa sección, y en este caso no es necesario cerrarla con `@endsection`; es lo que ocurre con la sección `titulo`.

Ahora nuestra vista de clientes podría quedar de la siguiente forma:

5.1. Envío de formularios con Laravel

Vamos a definir un formulario para dar de alta clientes en nuestro proyecto de **Gestión**. Creamos una vista llamada **alta.blade.php** en la subcarpeta **resources/views/clientes**:

```
<h1>Alta de cliente</h1>

<form action="{{ route('clientes.insertar') }}" method="post">

    <div class="form-group">
        <label for="nombre">Nombre:</label>
        <input class="form-control" type="text" name="nombre" id="nombre">
    </div>
    <div class="form-group">
        <label for="apellidos">Apellidos:</label>
        <input class="form-control" type="text" name="apellidos" id="apellidos">
    </div>

    <div class="form-group">
        <label for="dni">DNI:</label>
        <input class="form-control" type="text" name="dni" id="dni">
    </div>

    <input class="btn btn-dark btn-block" type="submit" value="Enviar"
        name="enviar" >

</form>
```

De forma predeterminada, Laravel protege de ataques XSS (*cross-site scripting*) de suplantación de identidad, por lo que obtendremos un error de tipo 419 si enviamos un formulario no validado. Para ello, utilizamos en el formulario la directiva `@csrf`, que añade un campo oculto con un *token* de validación del usuario.

Llamada desde un formulario a un método del controlador

```
<form action="{ route('clientes.insertar) }" method="post">
    @csrf
    ...
</form>
```

Con esto estamos enviando los datos del formulario a la ruta **clientes**, método **alta**, garantizando un mínimo de seguridad frente a ataques XSS (*cross-site scripting*) de suplantación de identidad, gracias a la directiva **@csrf** (*cross-site request forgeries*).

Definimos la ruta de clientes en **routes/web.php**:

```
Route::get('clientes/alta', [ClienteController::class, 'alta'])->name('clientes');
```

Cuando en la URL pongamos **http://localhost/clientes/alta**, llamará al método **alta** dentro del controlador **Cliente**, situado en la carpeta **Http/Controllers**.

```
public function alta()
{
    //return "Llamando al método alta() desde el controlador Cliente";
    return view('clientes.alta');
}
```

Ahora debemos recoger todos los campos del formulario. Si volvemos a la definición de este, podemos observar que el atributo **«action»** apunta al método **insertar** del controlador **Cliente**, por lo que definiremos una nueva ruta que atienda peticiones POST en **routes/web.php**.

```
Route::post('clientes/insertar',
[ClienteController::class, 'insertar'])->name('clientes.insertar');
```

En el *controller* **Cliente** debemos definir otro método **insertar**, que recibirá un objeto de tipo **Request**, donde obtendremos todos los campos del formulario. Recordemos que en PHP nativo los obtendríamos con **\$_POST['nombre_campo']**.

```
public function insertar(Request $request)
{
    $nombre = $request->get('nombre');
    $apellidos = $request->get('apellidos');
    $dni = $request->get('dni');

    return redirect()->route('clientes.index');
}
```


5.2. Validación de formularios

Además de validar a nivel de HTML los datos en el cliente, también se deben validar los datos en el servidor. Para la realización de esta acción, disponemos en el objeto **request** de un método **validate**, al que pasamos un *array* con las reglas de validación.

Por ejemplo, antes de llevar a cabo ninguna acción, comprobaríamos que el nombre y los apellidos tienen un tamaño mínimo, para evitar datos inconsistentes en nuestra base de datos. Si la validación falla, este método lanzará una excepción de tipo **Illuminate\Validation\ValidationException**, y el propio *framework* se encargará de devolver un mensaje de redirección al navegador para que acceda a la URL que teníamos cargada previamente.

```
public function insertar(Request $request) {
    request()->validate([
        [
            'nombre' => 'required|min:4',
            'apellidos' => 'required|min:4',
            'dni' => 'required|min:9'
        ], [
            'nombre.required' => 'El nombre es obligatorio',
            'apellidos.required' => 'El apellido o apellidos es obligatorio',
            'dni.required' => 'El DNI es obligatorio y debe tener 9 caracteres como mínimo',
        ]
    ]);

    // Código para procesar el formulario

}
```

En varios campos se han añadido dos o más reglas de validación enlazadas por la barra de *pipe* |. Toda la documentación relacionada con las reglas de validación disponibles se puede encontrar en **available-validation-rules**, en <https://bit.ly/validation-laravel>

A. Form requests para validaciones más complejas

Existe otra alternativa para validaciones más complejas, de forma que así evitamos poner más código en los controladores y organizamos mejor el código. Se trata de los *form request*, una clase adicional que contiene toda la lógica de validación. Los creamos con el comando **php artisan**.

```
php artisan make:request ClientePost
```

Esta clase se almacena de forma predeterminada en **app/Http/Requests**, y contiene los siguientes métodos predefinidos:

Authorize: devolvemos booleano, dependiendo de si el usuario está autorizado a enviar la petición o no. Si no se requiere autorización, devolveremos **true**.

Rules: para indicar las reglas de validación.

Messages (opcional): para personalizar los mensajes de error.

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class ClientePost extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array
     */
    public function rules()
    {
        return [
```

```

        'nombre' => 'required|min:4',
        'apellidos' => 'required|min:4',
        'dni' => 'required|min:9'
    ];
}

public function messages()
{
    return [
        'nombre.required' => 'El nombre es obligatorio',
        'apellidos.required' => 'El apellido o apellidos es obligatorio',
        'dni.required' => 'El DNI es obligatorio y debe tener 9
                           caracteres como mínimo',
    ];
}
}

```

Con esta alternativa en el método del controlador, tendremos que inyectar este **form_request** como parámetro. La validación es automática; si no se cumplen las reglas, nunca llegará a ejecutarse el código del método.

```

public function insertarConValidacion(ClientePost $request) {
    //Si entramos aquí, el formulario es válido.
}

```

B. Mostrar mensajes de error

Si la validación falla para algún campo, retornaremos a la página del formulario, notificando la información del error. Desde cualquier lugar de Laravel podemos acceder a la variable **\$errors**; esta variable tiene un método **any()**, que comprueba si hay algún error previo. Tenemos otro método, **all()**, que nos devuelve un *array* con los errores producidos. Aquí tenemos un ejemplo de cómo mostraríamos el listado de errores encima del formulario.

```

@if ($errors->any())
    <ul>
        @foreach($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
    </ul>

@endif

```

Disponemos de otros métodos, como **has('nombre-campo')** o **first()**, para saber si un campo tiene errores asociados y obtener el primer error.

```
<form action="{{ route('clientes.alta') }}" method="post">
    @csrf
    <div class="form-group">
        <label for="nombre">Nombre:</label>
        <input class="form-control" type="text" name="nombre" id="nombre">
        @if ($errors->has('nombre'))
            <div class="text-danger">
                {{ $errors->first('nombre') }}
            </div>
        @endif
    </div>
</form>
```

En definitiva, conseguiremos mostrar mensajes de error para los campos que hayan dado errores al validar.

C. Recordar valores enviados

Cuando validamos el formulario, si falla algún campo y tenemos que volver atrás, resulta muy tedioso tener que volver a rellenar todos los campos otra vez; para ello, **blade** nos ofrece la posibilidad de mostrar el valor antiguo en el atributo **value**, con una función llamada **old('nombre-campo')**; por ejemplo:

```
<input class="form-control" type="text" name="nombre" id="nombre"
value="{{ old('nombre') }}">
```

En el caso de **textarea**:

```
<textarea name="mensaje" id="mensaje" cols="30" rows="10">{{
old('mensaje') }}</textarea>
```



VÍDEOLAB

Para consultar el siguiente vídeo sobre página de inicio, escanea el código QR o [pulsa aquí](#).

UAX FP